

analytical_exercises/rnn_6d.py

```

1  #!/usr/bin/env python3
2  """
3  Problem 6(d) - Minimal RNN trainer for a fixed sequence 101010.
4
5  RNN model:
6       $s_k = w_{\text{rec}} * s_{k-1} + w_x * x_k$ 
7       $y = s_T$ 
8       $L = (t - y)^2$ , where  $t$  = number of 1s in the sequence.
9  """
10
11 import os
12 import random
13 from typing import List, Tuple
14
15
16 # -----
17 # Forward pass: compute all states and final output y
18 # -----
19
20 def forward_rnn(seq: List[int], w_rec: float, w_x: float) -> Tuple[float, List[float]]:
21     """
22     Forward pass for a single sequence.
23
24     Args:
25         seq: list of bits (0/1), e.g. [1, 0, 1, 0, 1, 0]
26         w_rec: recurrent weight
27         w_x: input weight
28
29     Returns:
30         y: final output ( $s_T$ )
31         states: list [ $s_0, s_1, \dots, s_T$ ] for use in backprop
32     """
33     s = 0.0 # initial state  $s_0$ 
34     states = [s] # store  $s_0$ 
35
36     for x in seq:
37         # recurrence:  $s_k = w_{\text{rec}} * s_{k-1} + w_x * x_k$ 
38         s = w_rec * s + w_x * x
39         states.append(s)
40
41     y = s # final state is the output
42     return y, states
43
44
45 # -----
46 # Backward pass: BPTT for this simple 1-D RNN
47 # -----
48
49 def backward_rnn(
50     seq: List[int],
51     t: int,

```

```

52     states: List[float],
53     w_rec: float
54 ) -> Tuple[float, float]:
55     """
56     Backpropagation through time for a single sequence.
57
58     Uses:
59          $L = (t - y)^2$ ,  $y = s_T$ 
60          $\delta_T = dL/ds_T = 2(y - t)$ 
61          $\delta_k = w_{rec} * \delta_{k+1}$ 
62          $\partial L / \partial w_x = \sum_k \delta_k x_k$ 
63          $\partial L / \partial w_{rec} = \sum_k \delta_k s_{k-1}$ 
64
65     Args:
66         seq:    input sequence (list of 0/1)
67         t:      target (number of 1s in seq)
68         states: list [s_0, ..., s_T] from forward pass
69         w_rec:  recurrent weight used in forward
70
71     Returns:
72         grad_w_rec, grad_w_x
73     """
74     T = len(seq)
75     y = states[-1]    # s_T
76
77     # delta_T = dL/ds_T
78     delta = 2.0 * (y - t)
79
80     grad_w_rec = 0.0
81     grad_w_x = 0.0
82
83     # Iterate backwards: k = T-1, ..., 0
84     # seq[k] corresponds to time step k+1
85     # states[k] is s_k
86     for k in reversed(range(T)):
87         x_k = seq[k]
88         s_prev = states[k]
89
90         # Grad contributions from time step k+1
91         grad_w_x += delta * x_k
92         grad_w_rec += delta * s_prev
93
94         # Propagate delta backwards in time:  $\delta_k = w_{rec} * \delta_{k+1}$ 
95         delta *= w_rec
96
97     return grad_w_rec, grad_w_x
98
99
100 # -----
101 # Train the RNN for a single random initialization
102 # -----
103
104 def train_one_run(
105     seq: List[int],

```

```

106     t: int,
107     epochs: int,
108     eta: float,
109     init_seed: int
110 ) -> Tuple[float, float, float, float, float, float]:
111     """
112     Train the RNN on a single fixed sequence for a given number of epochs.
113
114     Args:
115         seq:         fixed input sequence
116         t:           target = number of 1s in seq
117         epochs:      number of epochs (here: 30)
118         eta:         learning rate
119         init_seed:   seed for random initialization
120
121     Returns:
122         (w_rec_init, w_x_init, initial_loss,
123          w_rec_final, w_x_final, final_loss)
124     """
125     rng = random.Random(init_seed)
126
127     # Random initialization of weights in a small range
128     w_rec = rng.uniform(-0.5, 0.9)
129     w_x   = rng.uniform(-0.5, 0.9)
130
131     # Compute initial loss before any update
132     y0, states0 = forward_rnn(seq, w_rec, w_x)
133     initial_loss = (t - y0) ** 2
134     w_rec_init, w_x_init = w_rec, w_x
135
136     # Training loop: same sequence, repeated for 'epochs' steps
137     for _ in range(epochs):
138         # Forward pass
139         y, states = forward_rnn(seq, w_rec, w_x)
140         loss = (t - y) ** 2
141
142         # Backward pass (BPTT)
143         grad_wr, grad_wx = backward_rnn(seq, t, states, w_rec)
144
145         # Gradient descent update
146         w_rec -= eta * grad_wr
147         w_x   -= eta * grad_wx
148
149     # Final loss after training
150     y_final, _ = forward_rnn(seq, w_rec, w_x)
151     final_loss = (t - y_final) ** 2
152
153     return w_rec_init, w_x_init, initial_loss, w_rec, w_x, final_loss
154
155
156 # -----
157 # Main: fixed sequence 101010, 30 epochs, save aligned table
158 # -----
159

```

```

160 def main() -> None:
161     # Fixed input sequence 101010
162     seq = [1, 0, 1, 0, 1, 0]
163     t = sum(seq)          # target = number of 1s = 3
164
165     epochs = 15
166     eta = 0.01           # learning rate (can be adjusted)
167
168     # We will use 3 different random initializations
169     init_seeds = [2, 16, 20]
170
171     # Ensure results directory exists
172     os.makedirs("results", exist_ok=True)
173     output_path = os.path.join("results", "rnn_6d_results.txt")
174
175     with open(output_path, "w", encoding="utf-8") as f:
176         # General info header
177         f.write("Problem 6(d) - RNN counting ones\n")
178         f.write(f"Input sequence: {seq}\n")
179         f.write(f"Target (number of 1s): {t}\n")
180         f.write(f"Epochs: {epochs}\n")
181         f.write(f"Learning rate: {eta}\n\n")
182
183         # Table header (aligned columns)
184         header = (
185             f"{'Run':>3} | "
186             f"{'Seed':>4} | "
187             f"{'w_rec_init':>12} | "
188             f"{'w_x_init':>10} | "
189             f"{'initial_loss':>13} | "
190             f"{'w_rec_final':>12} | "
191             f"{'w_x_final':>10} | "
192             f"{'final_loss':>11}\n"
193         )
194         f.write(header)
195         f.write("-" * (len(header) - 1) + "\n") # separator line (minus final \n)
196
197         # One row per run
198         for run_id, seed in enumerate(init_seeds, start=1):
199             (
200                 w_rec_init,
201                 w_x_init,
202                 initial_loss,
203                 w_rec_final,
204                 w_x_final,
205                 final_loss,
206             ) = train_one_run(
207                 seq=seq,
208                 t=t,
209                 epochs=epochs,
210                 eta=eta,
211                 init_seed=seed,
212             )
213

```

```
214         row = (  
215             f"{run_id:>3} | "  
216             f"{seed:>4} | "  
217             f"{w_rec_init:12.6f} | "  
218             f"{w_x_init:10.6f} | "  
219             f"{initial_loss:13.6f} | "  
220             f"{w_rec_final:12.6f} | "  
221             f"{w_x_final:10.6f} | "  
222             f"{final_loss:11.6f}\n"  
223         )  
224         f.write(row)  
225  
226     print(f"Results written to: {output_path}")  
227  
228  
229 if __name__ == "__main__":  
230     main()  
231
```